

OBJECT ORIENTED PROGRAMMING (25MC409)

I M.C.A II Semester

2025-26 EVEN SEMESTER

**Name of the Faculty : Dr. K. Gayatri
Assistant Professor, Department of CA
VFSTR (Deemed to be University)**

Institute Vision & Mission

VISION:

To evolve in to a centre of excellence in Science & Technology through creative and innovative practices in teaching-learning, towards promoting academic achievement and research excellence to produce internationally accepted, competitive and world class professionals who are psychologically strong & emotionally balanced imbued with social consciousness & ethical values.

MISSION:

To Provide High Quality academic programmes, training activities, research facilities and opportunities supported by continuous industry - institute interaction aimed at promoting employability, entrepreneurship, leadership and research aptitude among students and contribute to the economic and technological development of the region, state and nation.

Department Vision & Mission

VISION:

To emerge as a renowned center in the field of computer applications for providing high quality education and research to produce globally competent professionals with ethical values.

MISSION:

- **M1:** To provide high quality education with cutting-edge curriculum and learner-centered approach.
- **M2:** To establish the culture of innovation and research through industry-academia linkages.
- **M3:** To cultivate ethical and socially-conscious professionals with leadership qualities, who understand the global implications of technology.

PO' s of the Department

- **PO1 (Foundation Knowledge):** Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.
- **PO2 (Problem Analysis):** Identify, review, formulate and analyze problems for primarily focusing on customer requirements using critical thinking frameworks.
- **PO3 (Development of Solutions):** Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
- **PO4 (Modern Tool Usage):** Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO' s of the Department

- **PO5 (Individual and Teamwork):** Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
- **PO6 (Project Management and Finance):** Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.
- **PO7 (Ethics):** Commit to professional ethics in managing software projects with financial | aspects. Learn to use new technologies for cyber security and insulate customers from malware
- **PO8 (Life-long learning):** Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

PEO' s of the Department

- **PEO1:** Develop cutting-edge technical skills to succeed in career and as an entrepreneur.
- **PEO2 :** Pursue higher education and research with critical thinking and problem solving capabilities
- **PEO3:** Demonstrate ethical, collaborative, and leadership abilities in multi-disciplinary domains.

PSO's of the Department

- PSO1: To provide solutions for complex computational problems with requisite mathematical, programming languages and tools.
- PSO2: To design, develop and analyse software and related services using algorithms, web design, and Big Data Analytics for real time applications.

Module-1 Syllabus

Unit-1

INTRODUCTION: Java buzzwords, OOP principles, Data types, Operators, Control statements, Type conversion and casting, Arrays. String class, String Tokenizer

Unit-2

CLASSES AND METHODS: Introduction to classes and methods, objects, Constructors, Overloading Methods and Constructors, Usage of static, Access control, this key word, Garbage Collection.

INHERITANCE: Basics of Inheritance, Types of inheritance, Abstract classes, Interfaces, Usage of final, Usage of super key word, implementing interface, extending interfaces.

Module-2 Syllabus

UNIT – 1

PACKAGES: Creating, defining and accessing packages, importing packages.

EXCEPTION HANDLING: Concepts of exception handling, Types of exceptions, Built-in exceptions, Usage of try, catch, throw, throws and finally keywords, Custom Exception.

UNIT – 2

MULTITHREADING: Thread, Thread life cycle, Thread creation, Thread priorities, multithreading, Thread Synchronization, Deamon Thread.

UNIT - 3

AWT, APPLET'S AND GUI PROGRAMMING WITH SWING: Applet life cycle, AWT, AWT Hierarchy, AWT Controls, Delegation Event Model.

EXPLORING SWING CONTROLS: JLabel, Jtext Field, JButton, JCheckBox, JRadioButton, JTabbed Pane, JList, JCombo Box.

Module-1

Unit-1

INTRODUCTION: Java buzzwords, OOP principles, Data types, Operators, Control statements, Type conversion and casting, Arrays. String class, String Tokenizer

Introduction

- A **programming language** is a formal language that specifies a set of instructions that can be used to produce various kinds of output.
- Programming languages generally consist of instructions for a computer.

Three major families of languages

- Procedural Programming Languages
- Structured Programming Languages
- Object Oriented Programming Languages

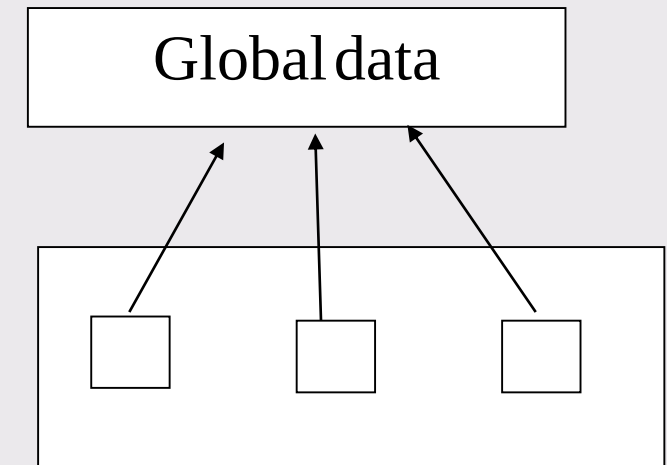
Procedural Programming Languages

- i) Emphasis on algorithm rather than data
- ii) Programs are organized in form of subroutine
- iii) Program controls are through jumps (goto) and calls to subroutine
- iv) Data move overly around the system from function to function
- v) It follows top down approach in program design
- vi) It is suitable for medium size software applications

Drawback:

- 1. Data Security problem
- 2. Difficult to maintain program code
- 3. It doesn't model to real world problem very well

Examples: Fortran, Cobol



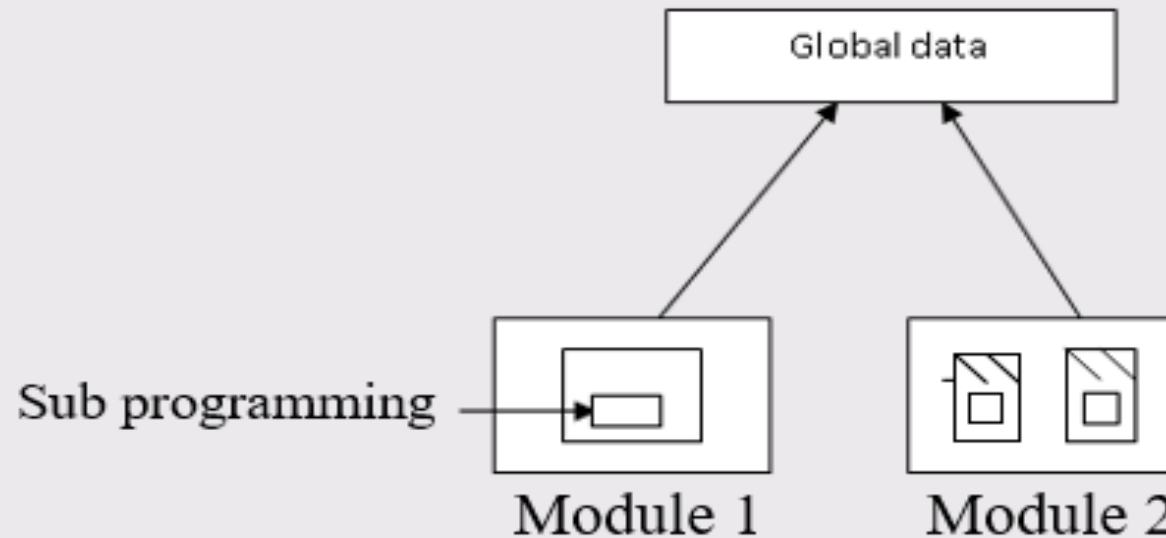
Structured Programming Languages

In Structured Programming Languages programs consist of multiple modules and each module has a set of functions of related types. The important features of Structured Programming Languages are

- i) Programs are divided into individual procedures that performs discrete tasks
- ii) Procedures are independent of each other
- iii) Procedures have their own local data and processing logic
- iv) Introduction of the concepts of user defined datatypes
- v) Maintenance of a large software system costly

Drawback: 1. Data Security problem
2. It doesn't model real world problem very well

Examples: Pascal, C

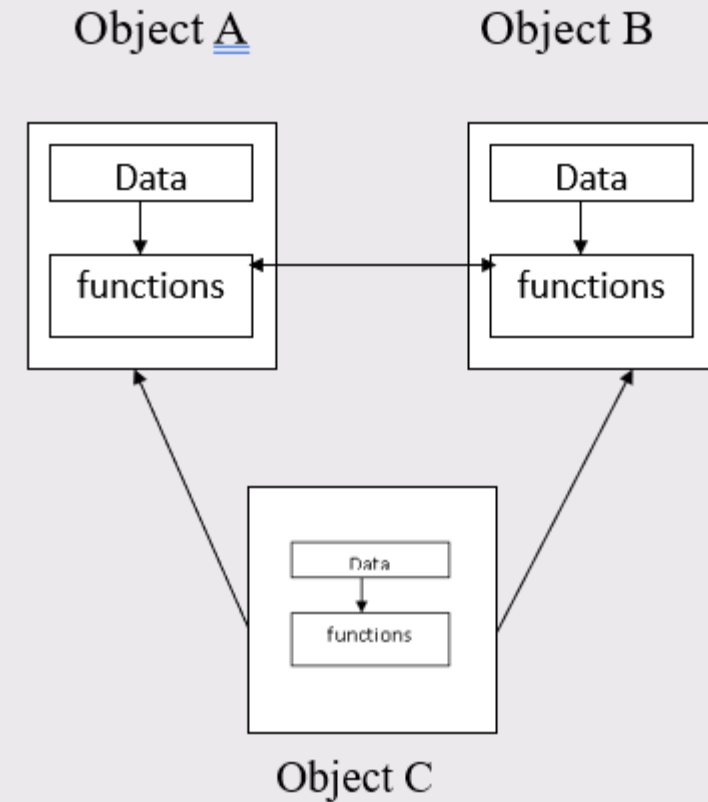


Object Oriented Programming Languages

The important features of Object Oriented Programming Languages are

- ❑ Emphasis on data rather than functions
- ❑ Programs are divided into objects
- ❑ Functions that operate on the data of an object are together in the data structure
- ❑ Data is hidden and cannot be accessed by external functions (Data security)
- ❑ Objects may communicate with each other through functions

- ❑ New data can be easily added to the functions whenever necessary
- ❑ It follows bottom-up approach in program designing





Introduction to JAVA

What is JAVA?





In today's era the internet and smart phones have become an integral part of our lives. We use our phones for almost all of our day-to-day tasks.

Earlier we used to shop at malls, queue up in front of banks and make our travel booking at travel agencies. But now, we can do all these in the comfort of our homes by using our phones. All of these are possible because of a high-level programming language called **JAVA**.

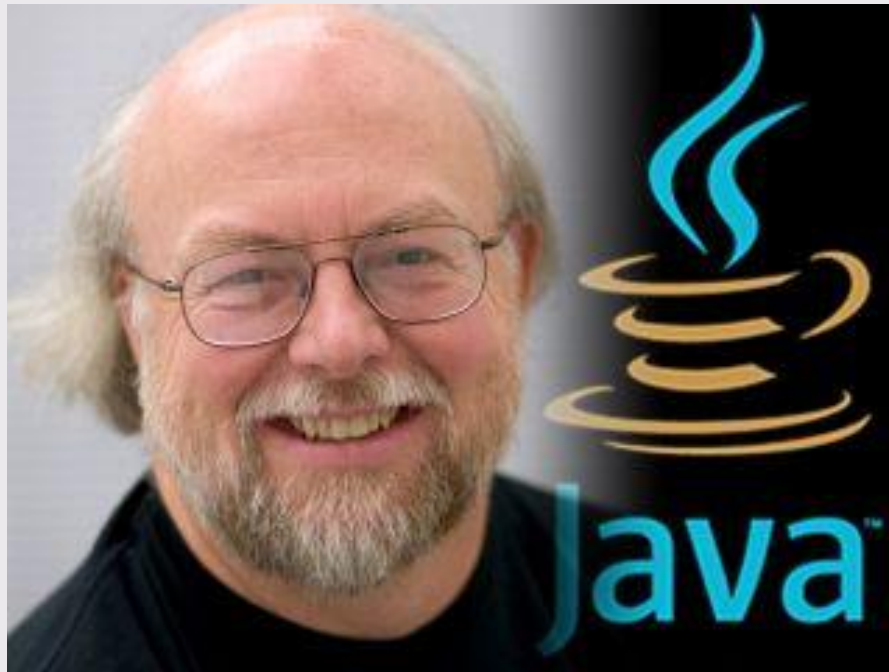
Java is an **Object Oriented** and **Class Based** Programming language that is used in a distributed environment on the internet. It is a **high-level** programming language which is easy to read and understand.



History of Java



Java is a computer-based programming language invented by **James Gosling** and **Sun Microsystems** in 1991.



Oracle Corporation is the current owner of the official implementation of the Java, following their acquisition of Sun Microsystems on **January 27, 2010**.



James Gosling had a **single motto** while creating this language

WRITE ONCE, RUN ANYWHERE

Which means that the code would have to be written only once
and can be used anywhere.

Editions of JAVA

- ☐ Java Standard Edition SE
- ☐ Java Enterprise Edition EE
- ☐ Java Micro Edition ME

There are many java versions that has been released.

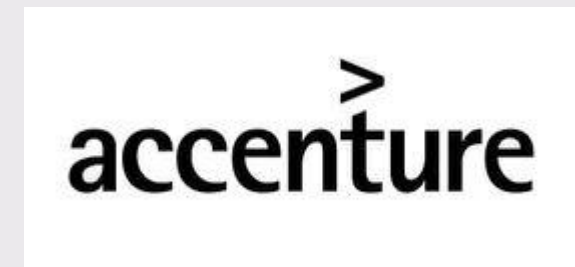
Current stable release of Java is Java SE 10.

- JDK Alpha and Beta (1995)
- JDK 1.0 (23rd Jan, 1996)
- JDK 1.1 (19th Feb, 1997)
- J2SE 1.2 (8th Dec, 1998)
- J2SE 1.3 (8th May, 2000)
- J2SE 1.4 (6th Feb, 2002)
- J2SE 5.0 (30th Sep, 2004)
- Java SE 6 (11th Dec, 2006)
- Java SE 7 (28th July, 2011)
- Java SE 8 (18th March, 2014)
- Java SE 9 (21st Sep, 2017)
- Java SE 10 (20th March, 2018)
- Java SE 14 - 2020

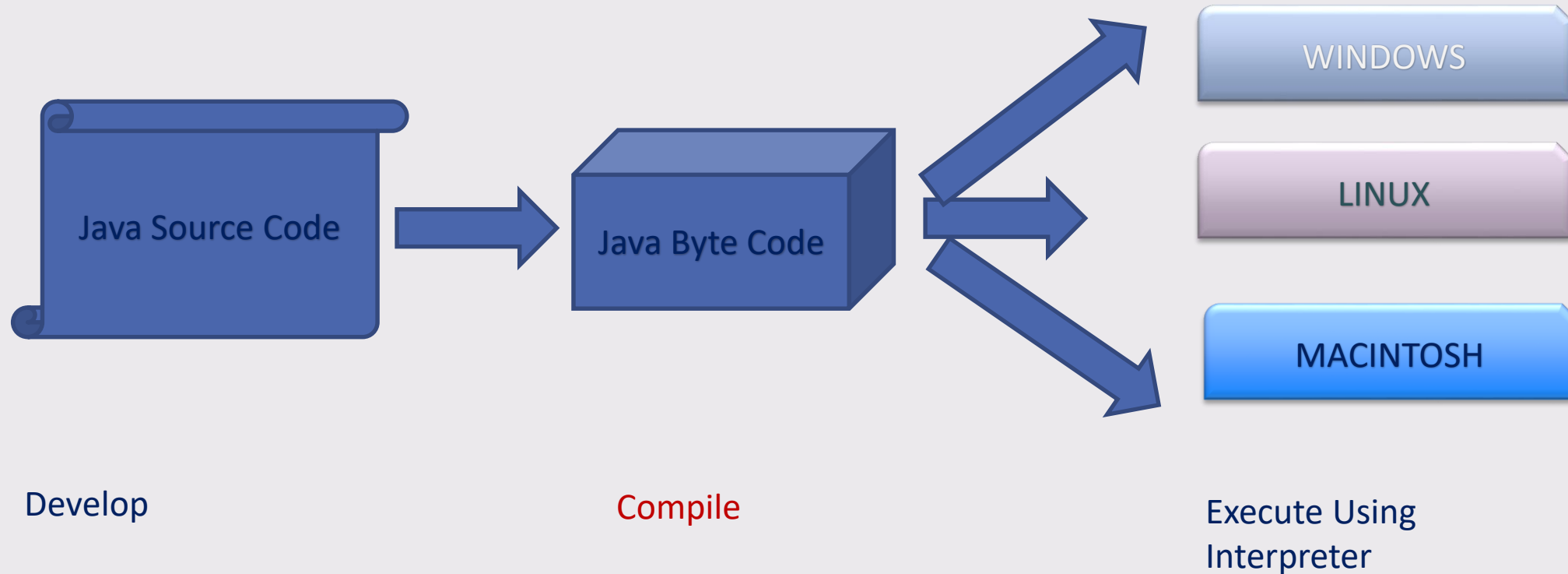
JAVA Based Products



JAVA Based Products



Java Program Execution





Byte Code

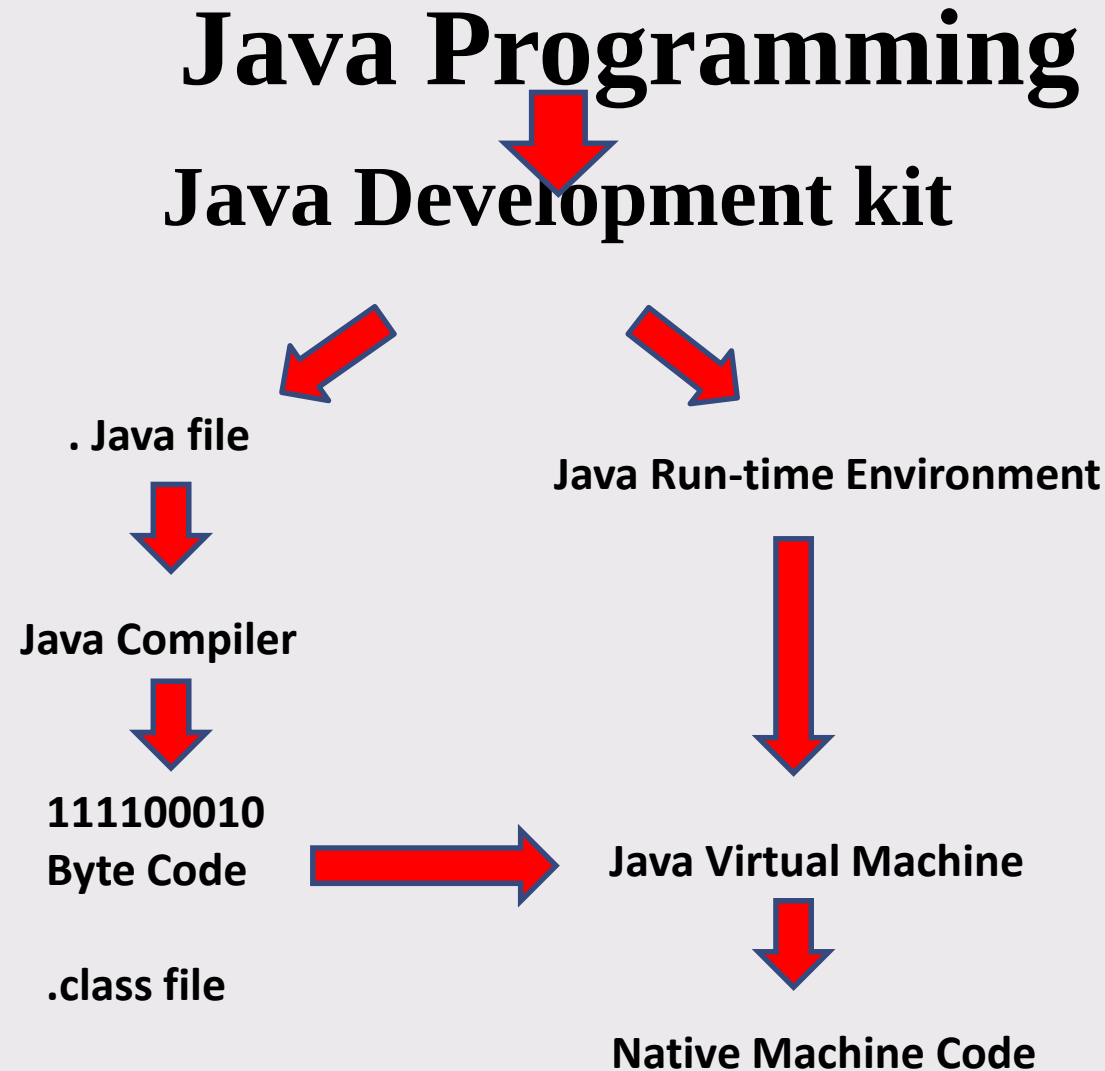


A **Byte code** is a program code that has been compiled from the source code into low level code.

Only **JVM** can understand this **byte code**. For every Operating System, it has its own JVM. Once the JVM got the byte code, it is going to convert that byte code into Native machine Code.

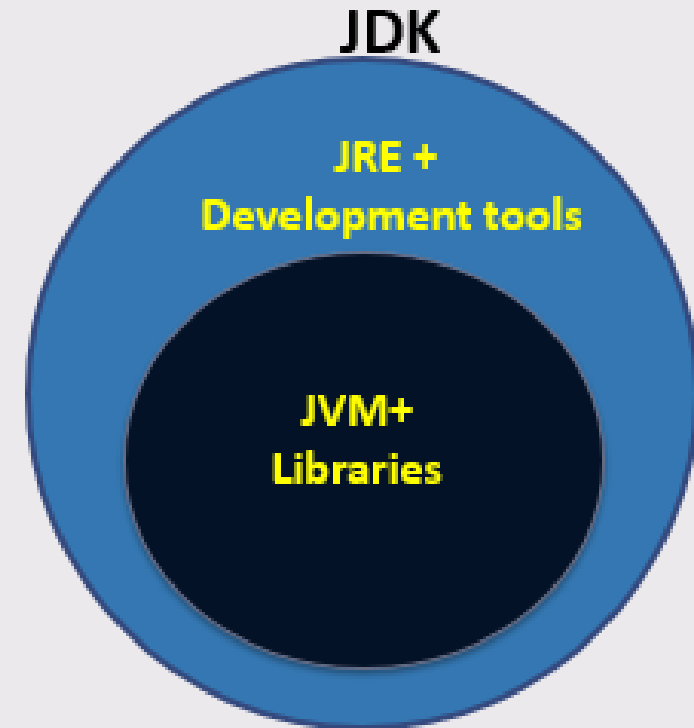
Based on the **operating System**, it is going to generate the corresponding the **Native Machine Code** and this code is passed to the **OS** for **execution**.

Java Execution Process (BYTE CODE)





JDK vs JRE vs JVM





- **JDK:** to develop and run a java application, the required environment is Java Development kit
- **JRE:** Just to run the Java application, the required environment is Java Runtime Environment.
- **JVM:** It is responsible for actually executing our Java Code. It executes our program line by line. So JVM is an interpreter.
- **Java** is both **compiled** and **interpreted** language.

Java Development Kit

- Java Development Kit is a collection of classes, Java compiler and Java virtual machine Interpreter. It also consist useful utilities for debugging, documentation, compiling and run java programs.
- The following are the some components of a Java Development Kit.
 - **javac:** The java compiler, converts java source code into byte code
Syntax: `javac filename.java`
 - **java:** The java interpreter, executes java application byte does directly from class file
Syntax: `java filename`

- **jdb**: The java debugger that allows to step through the program one line at a time, set break points, and examine variables

Syntax: jdb filename

- **servletrunner**: A simple web server to test servlets
- **appletviewer**: A java interpreter that executes java applet classes hosted by HTML

Syntax: appletviewer filename.html

Java Virtual Machine (JVM):

- Java provides both compiler and a software machine called JVM for each computer machine. The Java Compiler translates the Java Source Code (Java program) into an intermediate code known as byte code, which executes on special type of machine.
- This machine is called Java Virtual Machine and exists only inside the computer memory. The byte code is machine independent code and can run in any system.



- . The byte code is not a machine specific code. Java Interpreter takes the byte code and translates it into its own system machine language and runs the results.



Just In Time (JIT):

- The Java Virtual Machine is an interpreter that translates and runs each byte code instruction separately whenever it is needed by the computer program. In some cases it is very slow.
- As an alternative, Java also provides local compiler for each system that will compile byte code file into executable code for faster running. Java calls these compilers Just In Time compilers.

Java Standard Library(JSL)

The Java API (Application Programming Interface) is a collection of classes and methods grouped in the form of packages. JDK 1.2 has 58 packages, it is also known as Java Standard Library.

The important Packages in Java are

- a) **java.lang**: It contains the main language support class. It contains wrappers, strings and basic features of Java
- b) **java.util**: It provides classes that support date, time, basic event processing etc.,
- c) **java.io**: It provides reading and writing data in the form of streams.
- d) **java.awt**: It provides classes for creating GUI programs.
- e) **java.applet**: It includes set of classes to support for applet programming.
- f) **javax.swing**: It provides classes for swing operation programming.



Java Features (Buzz Words)



Following are the important features of java:

1. Simple
2. High performance
3. Secure
4. Robust
5. Portable
6. Dynamic
7. Distributed
8. Multithreaded
9. Object-oriented



- ❑ **Simple:** Java is pretty much simple language. Because Java is a high-level programming language. This high-level programming uses English like statements and symbols which we humans used to communicate. So, any one can read through it and understand what the code actually trying to do. Its context and style is easy to read and understand.
- ❑ **High Performance:** Java is an evolving system. The latest version of Java is released in March 2023, which provides a higher performance than the previous version. The Java bytecode was carefully designed so that it would be easy to translate directly into native machine code for very high performance.



❑ Secure:

- Java can be used to develop virus free systems.
- Java is secured because: Java program runs inside Virtual machine sandbox to prevent any activity from untrusted sources
- No use of explicit pointers

❑ Robust:

- Java checks the code during the compilation time and run time.
- Java completely takes care of memory allocation and releasing, which makes java more robust.



- ❑ **Portable:** Applications written on one platform of Java can be easily ported to another platform as it is platform independent. A portable language is one which can run on all operating systems and on all processors irrespective of their architectures.

- ❑ **Dynamic:** Many objects are evaluated at runtime and execution is carried out. Java programs carry with them substantial amounts of run-time type information that is used to verify and resolve accesses to objects at run time. This makes it possible to dynamically link code in a safe manner.



- ❑ **Distributed:** On internet information is distributed between various computers. Java is designed for the distributed environment of the Internet. In java, objects from two different computers allows to access methods remotely.
- ❑ **Multithreaded:**
 - Thread is a task in a process/program.
 - Multi-threading is multiple tasks running/ executing at the same time.
 - This facility is provided by Java so that many tasks can be executed at the same time.
- ❑ **Object Oriented:** Java is an object oriented programming language. Everything is performed using objects. Java can be easily extended as it is based on the Object model.

Object Oriented Programming System concepts

Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects. It simplifies the software development and maintenance by providing some concepts:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Object

- Any entity that has state and behavior is known as an object. For example: chair, pen, table, keyboard, bike etc. It can be physical and logical.
- Object can be defined as an instance of a class. An object contains an address and takes up some space in memory. Objects can communicate without knowing details of each other's data or code, the only necessary thing is that the type of message accepted and type of response returned by the objects.
- **Example:** A dog is an object because it has states i.e. color, name, breed etc. as well as behaviors i.e. wagging the tail, barking, eating etc.

Class

- *Collection of objects* is called class. It is a logical entity.
- A class can also be defined as a blueprint from which you can create an individual object. Class does not store any space.

Inheritance

- *When one object acquires all the properties and behaviors of parent object*, it is known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism

Polymorphism

- When one task is performed by different ways is known as polymorphism. For example: to convince the customer differently, to draw something e.g. shape or rectangle etc.
- In java, we use method overloading and method overriding to achieve polymorphism.
- Another example can be to speak something e.g. cat speaks meaw, dog barks woof etc.

Abstraction

- Hiding internal details and showing functionality is known as abstraction. For example: phone call, we don't know the internal processing.
- In java, we use abstract class and interface to achieve abstraction.

Encapsulation

- *Binding (or wrapping) code and data together into a single unit is known as encapsulation. For example: capsule, it is wrapped with different medicines.*
- *A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.*
- **Object Based Programming = Data Encapsulation + Data Hiding + Overloading (Polymorphism)**
- **Example: Ada**

Dynamic Binding

- When a method is called within a program, it associated with the program at run time rather than at compile time is called dynamic binding.
- **Object Oriented Programming = Object Based Programming Concepts + Inheritance + Dynamic Binding (Polymorphism)**
- **Example: C++, JAVA**

Java Tokens:

Tokens are smallest individual units in a program. The compiler reads the source program one character at a time grouping the source program into a sequence of atomic units called Tokens. Reserved words, identifiers, constants, operators etc., are the examples of Java Tokens.

Keywords:

The Keywords or Reserved words are one, which have a special and predefined meaning to the Java compiler. There are 48 reserved words currently used in Java. These words cannot be used as names for a variable, constant, class or method.

Reserved words are:

- abstract
- boolean
- break
- byte
- case
- catch
- char
- class
- const
- continue
- default
- do
- double
- else
- extends
- final
- Finally
- float
- for
- goto
- if
- implements
- import
- instanceof
- int
- interface
- long
- native
- new
- package
- Private
- protected
- public
- return
- short
- Static
- Strictfp
- super
- switch
- Synchronized
- this
- throw
- throws
- Transient
- try
- void
- volatile
- While
- true
- false
- null



Java Variable



A variable is nothing but a storage Container. A variable will holds the data

For example: age= 10

name= Gayatri

Variable is name of reserved area allocated in memory. In other words, it is a name of memory location. It is a combination of "vary + able" that means its value can be changed.



To create a variable, you must specify the type and assign it a value:

Syntax: Data_type variable_name = value;

Dynamic_Initialization

- Java allows variables to be initialized dynamically, using any expression valid at a time the variable is declared.

Example:

```
class sample
{
    public static void main(String args[])
    {
        double a=3.0, b=4.0;
        double c=Math.sqrt(a*a+b*b);
        System.out.println("Hypotenuse is" + c);
    }
}
```

Types of variables:



1) Local Variable

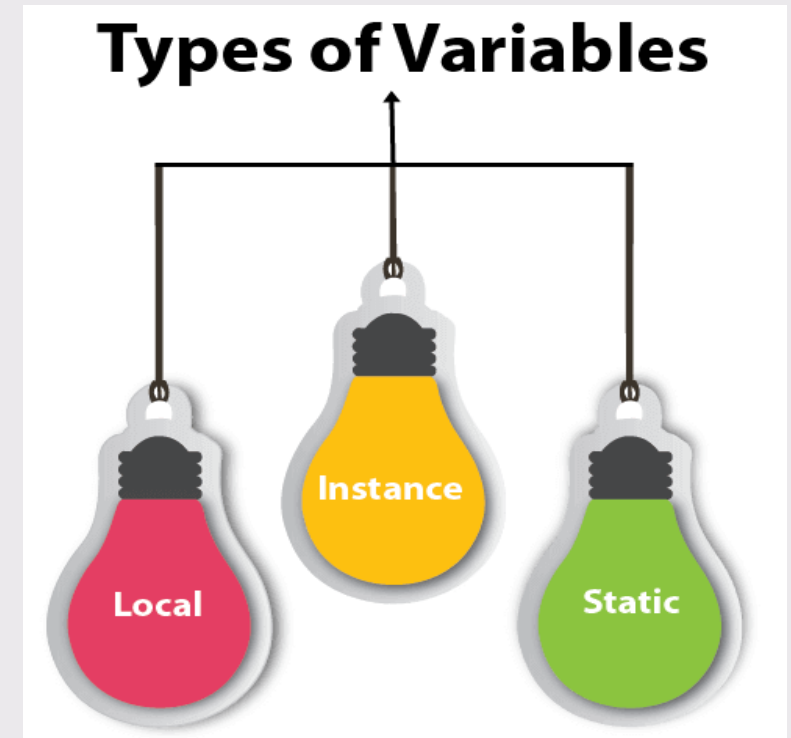
A variable declared inside the body of the method is called local variable. We can use this variable only within that method and the other methods.

2) Instance Variable

It is an Attribute of an Object. A variable declared inside the class but outside the body of the method, is called instance variable.

3) Static variable

A variable which is declared as static is called static variable. It cannot be local. Static variables belongs to Class



Scope & Life-time of a Variable

- An identifier duration also called lifetime, is the period during which that identifier exist in memory.
- Identifier that represent local variables in a method i.e., parameters and variables declared in the method body have automatic duration.
- Automatic declaration are created when program control reaches their declaration they exist while the block in which they are declare is active and they are destroyed when the block in which they are declared is exceeded. We will refer to variables of automatic duration has automatic variables or local variables.

- Java also has identifiers of static duration. Variables and references of static duration exist from the point at which the class that defines them is loaded into memory for execution until the program termination.
- Identifier scope is where the identifier can be a reference in a program. The scope of an identifier is class scope and block scope.
- Methods and instance variable of a class have class scope. Class scope begins at the beginning left brace of the class definition and ends at the closing right brace of the class definition.
- Identifiers declared inside a block have block scope. Block scope begins at the identifiers declaration and ends at the terminating right brace of the block. Local variables of the method have block scope.

Comments in a Program

Three types of comment statements are available in Java. They are

- Single line comment (//)
- Multi line comment (/* and */)
- Documentation comment (/** and */)

1. Single line comment: For a single line comment we use double slash (//) to begin a comment. It ends at the end of line.

Example: // Program for stack operations

2. Multi line comment: For more than one line of information, we use multi line comments. Multi line comments are starts with `/*` and ends with `*/`.

Example: `/* Program for stack operations
Designed by Arjun
VFSTR, GUNTUR */`

3. Documentation comment: For the documentation purpose, we use documentation comments. Documentation comments are starts with `/**` and ends with `*/`.

Example: `/** A test for JDK
@ Author K. Gayatri
@ version 1.0 **/`

Input and Output Statements

- java.lang package consists a class called 'System' which contains console I/O methods. If we want to take the information from the keyboard and display the information on the screen, Java uses Stream objects.
- The system provides three basic streams.
 - 1. **System.in**
 - 2. **System.out**
 - 3. **System.err**
- **System.in** refers to the standard input stream,
- **System.out** refers to the standard output stream and consists of **two methods** to print the strings and values on the screen.
 - 1. print()
 - 2. println().
- **System.err** refers to the standard error stream.

1. *print()*:

Syntax: `System.out.print(list);`

This statement prints the values of the variable name specified in the list of output unit.

Example: `System.out.print("hai");`

`System.out.print("java");`

O/P: hai java

2. *println()*:

Syntax: `System.out.println(list);`

This statement prints the values of the variable name in the list, the cursor is advanced by one line and the next statement is print in the next line.

Example: `System.out.println("hai");`

`System.out.println("java");`

O/P: hai java

Note: Java has a version of the '+' operator for string concatenation that enables a string and a value of another data type.

Example: `sum = 50`

`"sum is" + sum`

Here, `sum` is automatically converted to a string and concatenated with the `"sum is"` which results in the string `sum is 50`

Example: `"y#2="+7`

O/P: `y#2=7`

Compiling & Running the Java Program

- To compile Example.java program by the compiler javac, specify the name of the source file on the command line as

```
C:\> javac Example.java
```

- Here, the Java compiler creates a file called

Example: class that contain byte code version of the program. The class file is run by the Java interpreter, called java by passing the name to command line argument as

```
C:\> java Example
```

O/P: Hello



Data types

Data Types

- Java is *Strongly Typed Language*.
- *First, every variable has a type, every expression has a type, and every type is strictly defined.*
- Data types are used to specify the type of data that a variable can attain.
- There are 2 types of data types.
 1. Primitive Data types
 2. Non-Primitive Data types

Primitive Data type:

It is also called as standard, intrinsic or built-in data types. There are eight primitive data types available in Java. Those are byte, short, int, long, char, float, double and boolean.

These can be classified into four groups.

Integers	: byte, short, int, long
Floating point numbers	: float, double
Characters	: char
Boolean	: boolean

Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values

Note:

- 1. Java does not support have any unsigned types
- 2. Variables of types char, byte, short, int, long, float and double are all given the value '0' by default
- 3. Variable of type boolean are given 'false' by default
- 4. All primitive data types in Java are portable (Platform independent)

Non-Primitive Data type:

These are also known as Derived Data type or Abstract Data type or Reference type. These are built on primitive data types.

- Examples: string, class, array, interface etc.,



Naming Conventions

Rules to Declare a Variable



1. A variable name can consist of Capital letters A-Z, lowercase letters a-z digits 0-9, and two special characters such as _ underscore and \$ dollar sign.
2. The first character must not be a digit.
3. Blank spaces cannot be used in variable names.
4. Java keywords cannot be used as variable names.
5. Variable names are case-sensitive.
6. There is no limit on the length of a variable name but by convention, it should be between 4 to 15 chars.



Camel case in Java

There are two ways of using Camel case:

Lower camel case: where the first character of the first word is in lowercase. This convention is usually followed while naming the methods and variables.

Example: firstName, lastName, actionPerformed, printArray(), etc.

Upper camel: where the first character of the first word is in uppercase. This convention is usually followed while naming the classes and interfaces.

Example: Employee, Printable, etc.



Scenario: Siva owns a Retail Department Store. He needs to create a bill with the following fields present:

- ✓ Invoice ID
- ✓ Product ID
- ✓ Product Cost
- ✓ Quantity
- ✓ Discount
- ✓ Total Price
- ✓ Feedback Provided?



**Siva has no knowledge about data types in Java!
Help Mr. Siva!!!**



Program:1

Write a Java Program to add two numbers and print the result



```
class Sample
{
    public static void main(String args[])
    {
        int first = 10;
        int second = 20;
        int sum = first + second;
        System.out.println(first + " + " + second + " = " + sum);
    }
}
```



Program:2

Reading different types of data from the user
and display that data using Scanner class

```
import java.util.Scanner;
class HelloWorld
{
    public static void main(String args[ ])
    {
        // Using Scanner for Getting Input from User
        Scanner in = new Scanner(System.in);

        String s = in.nextLine( );
        System.out.println("You entered string "+s);

        int a = in.nextInt( );
        System.out.println("You entered integer "+a);

        float b = in.nextFloat( );
        System.out.println("You entered float "+b);
    }
}
```





Scenario: Prabhas has come to Siva's store to purchase a few products. Below are the items Prabhas wishes to purchase.

Product	Quantity	Cost
Item A	2	300
Item B	1	85
Item C	3	600

At the time Prabhas visited the store, there was a 10% discount on all products. And finally a service tax of 5% is applicable on total price.

Can you help Siva, compute the amount Prabhas has to pay?



Operators



Java operators can be divided into following categories:

- Arithmetic operators
- Assignment operators
- Comparison / Relational operators
- Logical operators
- Bitwise operators
- Conditional operator

Arithmetic Operators



Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value of a variable by 1	$++x$
--	Decrement	Decreases the value of a variable by 1	$--x$

Assignment Operators



Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3



Comparison Operators

Operator	Name	Example
<code>==</code>	Equal to	<code>x == y</code>
<code>!=</code>	Not equal	<code>x != y</code>
<code>></code>	Greater than	<code>x > y</code>
<code><</code>	Less than	<code>x < y</code>
<code>>=</code>	Greater than or equal to	<code>x >= y</code>
<code><=</code>	Less than or equal to	<code>x <= y</code>



Logical Operators

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	$x < 5 \ \&\& \ x < 10$
	Logical or	Returns true if one of the statements is true	$x < 5 \ \ x < 4$
!	Logical not	Reverse the result, returns false if the result is true	$!(x < 5 \ \&\& \ x < 10)$

Bitwise Operators



Operator	Description	Example
&	Bitwise AND	a & b
	Bitwise OR	a b
^	Bitwise exclusive OR	a ^ b
<<	left shift	a << 2
>>	right shift	a >> 2

Conditional operator



It is also known as ternary operator because it works with three operands.

`expr1 ? expr2 : expr3`

In ternary operator, if `expr1` is true then expression evaluates after question mark (?) else evaluates after colon (:)



Control Statements

There are four types of if statement in Java:



i.if statement

ii.if-else statement

iii.if-else-if ladder

iv.nested if statement

if statement: The if statement conditionally process the statements when the specified test condition is true.

Syntax: if(condition)
{
 // Block of statements
}

if-else statement: In if-else statement general form is as

Syntax: if (condition)
{
 //true block statements
}
else{
 //false block statements
}

Nested if-else statement: The if-else statement is placed within another if-else statement such methods are called nested if-else statement.

Syntax:

```
if (condition1)
{
    //block1 statements
}
else if(condition2)
{
    //block2 statements
}
else{
    //block3 statements
}
```

switch statement: The switch statement allows section among the multiple section of a code depending on the value of an expression. The objective is to check several possible constant values for an expression.

Syntax: switch (expression)

```
{  
    case value1:    block1 statements  
        break;  
    case value2:    block2 statements  
        break;  
    .  
    .  
    default:       default block statements  
}
```



Loops

Loop Control Statements:

A Loop statement allows to run a statement or block of statements repeatedly for a certain number of times. The repetition continues while the condition is true. When the condition becomes false, then loop ends and control passed to the next statements following the loop. Repetitive execution of statements is called looping. Java provides three types of loop control statements.

- while statement
- do-while statement
- for statement

while statement: while loop executes the statements repeatedly as long as a given condition becomes true. When the condition becomes false the control immediately pass to the next statements available after the loop statements.

Syntax: while (condition)
 {
 //statements
 }

do-while statement: In do-while loop statements, first the control executes the statements then it checks for the condition. The statements are executed repeatedly as long as a given condition becomes true. If the condition becomes false then the control comes out from the loop and executes the statements available after the loop statements.

Syntax: do
 {
 //statements
 } while (condition);

for statement: The for statement also repeat a statement or compound statements for a specified number of times. The general form of for statement is as

Syntax: for(initialization; condition; increment/decrement)

```
{  
  
    //statements  
  
}
```


Jump Statements

Jump Control Statements:

Java provides the following Jump Control statements.

- break statement
- continue statement
- return statement
- exit statement

break statement: break statement executed in a while, do-while, for or switch statements causes immediate exit from that structure. It can be used in two ways as unlabeled break and labeled break statement.

Unlabeled break statement:

Syntax: break;

Example:

```
for(int k=1;k<=10;k++)  
{  
    if(k==5)  
        break;  
    System.out.println(k);  
}
```

O/P: 1 2 3 4

Labeled break statement:

- To break out of nested structures, we can use the labeled break statements.
- This statement, when executed in a while, do-while and for statements causes immediate exit from that structure and any number of enclosing repetition structures.
- Program execution resumes with the first statement after the enclosing labeled compound statement.

```
class ex
{
public static void main(String[] args)
{
stop:
{
for(int row=1;row<=10;row++)
{
for(int col=1;col<=5;col++)
{
if(row==3)
break stop;
System.out.print("* ");
}

```

```
System.out.println("\n");
```

```
}
```

```
}
```

```
}
```

```
}
```

O/P: * * * * *

 * * * * *

continue statement: continue statement executed in a while, do-while or for statements skip the remaining statements in the body of that structure and proceeds with the next iteration of the loop. It can be used in two ways as unlabeled continue and labeled continue statement.

Unlabeled continue statement:

Syntax: continue;

Example:

```
for(int k=1;k<=10;k++)  
{  
    if(k==5)  
        continue;  
    System.out.println(k);  
}
```

O/P: 1 2 3 4 6 7 8 9 10

Labeled continue statement: The labeled continue statement when executed in a repetition structure skips the remaining statements in that structure body and any number of enclosing repetition on structure and proceeds with the next iteration of an enclosed labeled repetition.

Example:

```
class ex
{
    public static void main(String[] args)
    {
stop:
        for(int row=1;row<=4;row++)
        {
```

```
for(int col=1;col<=3;col++)  
{  
    if(col==2)  
        continue stop;  
    System.out.print(col);  
}  
System.out.println("\n");  
}  
}
```

O/P: 1111

- **return statement:** The return statement is used to explicitly return from a method. This causes program control to transfer back to the caller of the method. The return statement immediately terminates the method in which it executes.

Syntax: `return;`

- **exit statement:** The method `exit( )` is defined in `System` package. The purpose of `exit` is to terminate the program with a specific exit code.

Syntax: `System.exit(exit_code);`

The program finish normally by placing `exit_code` as '0' other value means error.

Example: `System.exit(0);`

- Entire class definition will be between the opening curly brace { and the closing curly brace }
- 'public' keyword is an access specifier, which allows the programmer to control the visibility of class members.
- The keyword 'static' allows main() to be called without having to instantiate a particular instance of the class. Since, main() method is called by the Java interpreter before any objects are made.
- The keyword 'void' used to tell the compiler that main() does not return a value.
- main() is a method called when a Java application begins.
- String args[] declares a parameter named args, which is an array of instances of the class string. args receives any command line arguments present when the program is executed.
- Now the file is saved with .java extension . Let the file name is taken as Example.java



Type conversion and casting

Converting one data type to another data type

Type conversion and Casting:

Java permits mixing of constants and variables of different types, but during execution it follows type conversions. In Java, type casting can be done in two ways. They are

1. Implicit Type Casting
2. Explicit Type Casting

1. Implicit Type Casting:

In an expression, if the operands are of different types the lower type is automatically converted to the higher type before the operation preceeds by the Java Interpreter.

The result is of the higher type. Such type of casting is called Implicit Type Casting.

In Implicit Type Casting there is no loss of data.

The following are some of implicit type castings with no loss of data.

From	To
byte	short, char, int, long, float, double
short	char, int, long, float, double
char	int, long, float, double
int	long, float, double
long	float, double
float	double

The general form of casting is as

Syntax: (datatype) expression;

Example: `x = (int) 7.5;`

Note: The final result of an expression is converted to the type of the variable before assigning to the left-hand side variable. In such case following chances are introduced.

- float to int causes truncation of the fractional part
- double to float causes rounding of digits
- long to int causes dropping of the excess of higher order bits

2. Explicit Type Casting:

- If we want to force to convert the operands of an expression from one type to another type depending on user choice, such type of casting is called Explicit Type Casting.
- In Explicit Type Casting there is a chance of data loss when we convert the operands from higher type to lower type.
- The process of converting the value is known as Casting a value.

Converting int to double

```
class Main {  
    public static void main(String[] args) {  
        // create int type variable  
        int num = 10;  
        System.out.println("The integer value: " + num);  
  
        // convert into double type  
        double data = num;  
        System.out.println("The double value: " + data);  
    }  
}
```

Converting double into an int

```
class Main {  
    public static void main(String[] args) {  
        // create double type variable  
        double num = 10.99;  
        System.out.println("The double value: " + num);  
  
        // convert into int type  
        int data = (int)num;  
        System.out.println("The integer value: " + data);  
    }  
}
```



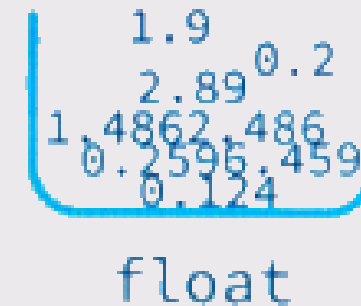
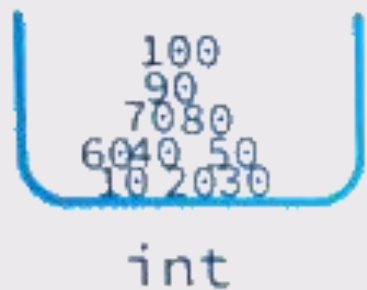
Array

Array

Array is used to store multiple values in a single variable, instead of declaring separate variables for each value.

Array is a collection of similar data elements.

Array is a collection of variables of the same data type.



To create an array:

datatype [] arrayname = new datatype[array size];

datatype [] arrayname = {elements};

int[] arr = new int[5];

arr[0] = 10;

arr[1] = 20;

arr[2] = 30;

arr[3] = 40;

arr[4] = 50;

int[] arr = {10,20,30,40,50};



```
class Sample
{
    public static void main(String args[])
    {
        int[] arr = new int[5];
        arr[0] = 10;
        arr[1] = 20;
        arr[2] = 30;
        arr[3] = 40;
        arr[4] = 50;

        for (int i = 0; i < arr.length; i++)
            System.out.println("Element at index " + i + " : " + arr[i]);
    }
}
```

One-Dimensional Array:

- One-Dimensional Array is a list of homogeneous items can be given one variable name using only one subscript.
- Individual value of an array is called an element. Like an ordinary variable, arrays must be declared and created in the computer memory before they are used.
- Creation of an array contains two sections.

Syntax: `type arrayname[ ];` `// declaration`

`arrayname = new type[size];` `// allocation of memory`

(or)

```
type arrayname[ ]=new type[size];
```

Where,

- Type defines the datatype of the variables stored in the array
- Array name defines the name of the array
- New operator is used to allocate dynamic memory in the computer for the array
- Size defines the number of memory location of the array

Example: `int number[];`

`number = new int[5];`

- It can also be initialize arrays automatically when they are declared.
The syntax is as

Syntax: `type arrayname[]={list of values};`

- Where, list of values contains number of values separated by commas and surrounded by curly braces

Example: `type number[]={35,40,56,23};`

- In Java, all arrays store the allocated size in a variable named length.
We can access the length of the array using the syntax

Syntax: `arrayname.length;`

```
class arr2
{
    public static void main(String[] args)
    {
        int sum=0;
        int n[]={3,4,6,7,8};
        for(int i=0;i<5;i++)
        {
            sum=sum+n[i];
        }
        System.out.println("Total="+sum);
    }
}
```

```
class arr1
{
    public static void main(String[] args)
    {
        int sum=0;
        int no[]=new int[10];
        for(int i=0;i<5;i++)
        {
            no[i]=Integer.parseInt(args[i]);
            sum=sum+no[i];
        }
        System.out.println("Total="+sum);
    }
}
```

```
import java.util.Scanner;
class arr2
{
    public static void main(String[] args)
    {
        int sum=0;
        int no[]=new int[10];
        Scanner sc= new Scanner(System.in);
        for(int i=0;i<5;i++)
        {
            System.out.println("enter elements");
            no[i]=sc.nextInt();
            sum=sum+no[i];
        }
        System.out.println("Total="+sum);
    }
}
```

Two-Dimensional Array:

A Two-Dimensional Array is a collection of homogeneous data items that share a common name using two subscripts.

It stores table of data. This representation is very useful for matrix representations as first subscript represents the number of rows and second subscript represents the number of columns.

Syntax: `type arrayname[][];` `// declaration`

`arrayname = new type[size1][size2];` `// allocation of memory`

(or)

```
type arrayname[ ][]=new type[size1][size2];
```

Where,

- Type defines the datatype of the variables stored in the array
- Arrayname defines the name of the array
- New operator is used to allocate dynamic memory in the computer for the array
- Size1 defines the number of rows of memory location of the array
- Size2 defines the number of columns of memory location of the array

- Example: `int number[][];`
`number = new int[5][6];`
- It can also be initialize arrays automatically when they are declared.
The syntax is as

Syntax: `type arrayname[][]={list of values};`

Or

```
type arrayname[][]={{row1 values},
                    {row2 values},
                    ..
                    }
```

- Eg: `int a[][]={{3,5,6,7},{5,6,8,7},{5,9,7,1}};`

```
public class TwoDimentionalArray {  
    public static void main(String[] args) {  
        int[][] a = { {15, 25, 35}, {45, 55, 65} };  
        int[][] b = { {12, 22, 32}, {55, 25, 85} };  
        int[][] Sum = new int[2][3];  
        int rows, columns;  
        for(rows = 0; rows < a.length; rows++) {  
            for(columns = 0; columns < a[0].length; columns++) {  
                Sum[rows][columns] = a[rows][columns] + b[rows][columns];  
            }  
        }  
    }  
}
```



```
System.out.println("Sum Of those Two Arrays are: ");  
for(rows = 0; rows < a.length; rows++) {  
    for(columns = 0; columns < a[0].length; columns++) {  
        System.out.format("%d \t", Sum[rows][columns]);  
    }  
    System.out.println("");  
}  
}  
}
```

Multi-Dimensional Array:

- Multi-Dimensional Array is a list of values shared by a common name using multi subscripts. If the array contains three subscripts then the array is called as Three-Dimensional array and so on.

Syntax:

```
type arrayname[][]...[]=new int[size1][size2]...[sizen];
```

Example: **type number[][][]=new int[4][5][6];**

- The Arrays class of the java.util package contains several static methods that we can use to fill, sort, search, etc in arrays. This class is a member of the Java Collections Framework and is present in **java.util.arrays**.

```
import java.util.Scanner;

public class ThreeDArray {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Declare 3D array
        int[][][] arr = new int[2][2][2];

        System.out.println("Enter elements for 3D array:");

        // Input elements
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                for (int k = 0; k < 2; k++) {
                    System.out.print("arr[" + i + "][" + j + "][" + k + "] = ");
                    arr[i][j][k] = sc.nextInt();
                }
            }
        }
    }
}
```

```
// Display elements
```

```
System.out.println("\n3D Array elements are:");
```

```
for (int i = 0; i < 2; i++) {
```

```
    for (int j = 0; j < 2; j++) {
```

```
        for (int k = 0; k < 2; k++) {
```

```
            System.out.print(arr[i][j][k] + " ");
```

```
        }
```

```
        System.out.println();
```

```
    }
```

```
    System.out.println();
```

```
}
```

```
sc.close();
```

```
}
```

```
}
```



String



In Java, a string is a sequence of characters or collection of characters

For example, "hello" is a string containing a sequence of characters 'h', 'e', 'l', 'l', and 'o'.

Different ways to create a String,

```
String s1 = "Gayatri";
```

```
String s2 = new String("Gayatri");
```



Java String Methods



charAt() method

- ✓ The charAt () method returns the character at the specified index in a string.

```
String myStr = " Gayatri";  
char result = myStr.charAt (6);  
System.out.println (result);
```




compareTo() method

- ✓ The compareTo () method compares two strings.

Example:

```
String myStr1 = "hello";
```

```
String myStr2 = "hello";
```

```
System.out.println (myStr1.compareTo (myStr2));
```



Concat() method

- ✓ The concat () method appends (concatenate) a string to the end of another string.

Example:

```
String firstName = "K ";
```

```
String lastName = " Gayatri";
```

```
System.out.println (firstName.concat (lastName));
```



endsWith() and startsWith() methods

- ✓ The `endsWith ()` method checks whether a string ends with the specified character(s).
- ✓ The `startsWith ()` method checks whether a string starts with the specified character(s).

Example:

```
String myStr = "Hello";  
System.out.println (myStr.startsWith ("Hel"));  
System.out.println (myStr.startsWith ("llo"));  
System.out.println (myStr.endsWith ("o"));
```



equals() method

- ✓ The equals () method compares two strings, and returns true if the strings are equal, and false if not.

Example:

```
String myStr1 = "Hello";  
String myStr2 = "Hello";  
String myStr3 = "Another String";  
System.out.println (myStr1.equals (myStr2));  
System.out.println (myStr1.equals (myStr3));
```



length() method

- ✓ The length () method returns the length of a specified string.
- ✓ Note: The length of an empty string is 0.

Example

```
String txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
System.out.println (txt.length ());
```



isEmpty() method

- ✓ The isEmpty () method checks whether a string is empty or not.
- ✓ This method returns true if the string is empty (length () is 0), and false if not.

Example

```
String myStr1 = "Hello";
```

```
String myStr2 = "";
```

```
System.out.println (myStr1.isEmpty ());
```

```
System.out.println (myStr2.isEmpty ());
```



indexOf() method

- ✓ The `indexOf ()` method returns the position of the first occurrence of specified character(s) in a string.

Example

```
String myStr = "Hello planet earth, you are a great planet.";
System.out.println (myStr.indexOf ("planet"));
```



replace() method

- ✓ The `replace ()` method searches a string for a specified character, and returns a new string where the specified character(s) are replaced.

Example:

```
String myStr = "MAD";
```

```
System.out.println (myStr. replace ('M', 'S'));
```




toLowerCase() and toUpperCase() methods

- ✓ The toLowerCase () method converts a string to lower case letters.
- ✓ The toUpperCase () method converts a string to upper case letters.

Example

```
String txt = "gayatri";  
String txt1 = "GAYATRI";  
System.out.println (txt.toUpperCase ());  
System.out.println (txt.toLowerCase ());
```



trim() method

✓ The trim () method removes whitespace from both ends of a string.

Example

```
String myStr = "    Gayatri    ";  
System.out.println(myStr);  
System.out.println(myStr.trim());
```



String Tokenizer



In Java, `StringTokenizer` is used to break a string into tokens based on provided delimiter.

The `java.util` package has a class called `StringTokenizer` that is used to tokenize strings.

Following are the constructors in string tokenizer

```
StringTokenizer(String str)  
StringTokenizer(String str, String delim)
```



Methods of StringTokenizer

Method	Description
boolean hasMoreTokens()	Return True if more token are available else return False.
String nextToken()	Return next token of StringTokenizer Object
String nextToken(String delim)	Return Next token based on delimiter.
int countTokens()	count Total number of Token



```
import java.util.StringTokenizer;
public class Sample
{
    public static void main(String args[])
    {
        StringTokenizer st = new StringTokenizer("Vignan University, Vadlamudi, Tenali");
        while (st.hasMoreTokens())
        {
            System.out.println(st.nextToken());
        }
    }
}
```



```
import java.util.StringTokenizer;
public class Sample
{
    public static void main(String args[])
    {
        String fruits = "apple,banana,orange,grape";
        StringTokenizer st = new StringTokenizer(fruits, ",");
        while (st.hasMoreTokens())
        {
            System.out.println(st.nextToken());
        }
    }
}
```



Thank you